A Causality-Preserving, Stereographically Compactified Worldline Representation for Retarded Liénard–Wiechert Electrodynamics

Tim Huff
Independent Researcher (Creve Coeur, MO, USA)
Software: hopf-hough v6.2.0, **[TODO: Zenodo software DOI for v6.2.0]** (MIT License)

Version 6.2 — October 2026

Abstract

We describe and verify `hopf-hough`, an open-source browser tool and library that evaluates the exact retarded Liénard–Wiechert (LW) field of moving point charges at every pixel of an observation plane on the GPU, interactively in a browser, and marks the pixels where the result should not be trusted. The mathematics is established; the contribution is a verified single-precision implementation. Each pixel solves, for every source, the light-cone equation $f(R) = R - \|\mathbf{x} - \mathbf{z}(R)\| = 0$ on a compact per-source history and evaluates the LW field from the interpolated history. Three ingredients make it accurate in WebGL2 float32: (i) the history is resampled onto shells of retarded radius placed by $R = L\tan(\theta/2)$; (ii) cubic Hermite segments are limited by a closed-form certificate on the Bernstein coefficients of $\|\mathbf{z}'(R)\|^2$, which guarantees $\|\mathbf{z}'\| \le \beta_{\max} < 1$ and hence one retarded root per observer; (iii) the root function is evaluated in double-float arithmetic at the bracketing knot only, and inside the segment by a cancellation-free incremental form that takes the chord deficit $1 - |\bar{\mathbf{v}}|^2$ from float64 values computed on the CPU. Version 6.1 attributed the remaining float32 error of the version-6 kernel to knot-level root-function arithmetic. At $\gamma \gtrsim 70$ it comes instead from the float32 offset inside the segment. The new default kernel, v6f, removes that floor; an optional kernel, v6f precise, also removes a smaller float32 floor from the direction terms of the field. Against the exact field (no softening), the relative L_2 error of v6f across a synchrotron pulse at $M_T = 512$ is 0.0076% at $\gamma = 22.4$ and 0.023% at $\gamma = 70.7$ (float32 observer positions, compared at the rounded point), and 0.066% at $\gamma = 224$ with double-float observer coordinates (0.0060% at $M_T = 2048$). On exactly sampled preset paths it is at most 1.4×10^{-3}% up to $\gamma = 70.7$, and 1.4×10^{-4}% with v6f precise. On an Apple M1 Max (Safari, Metal), v6f renders as fast as the version-6 kernel and v6f precise at $0.77\times$ its frame rate, and the in-app suite, run with both, matches the float64 evaluation of the same spline to the digits it prints. The double-precision reference agrees with PyCharge to $\sim 10^{-11}$ up to $\gamma = 7.1$. In far-field tests no observer left unmarked by the validity indicator is more than 0.68% from the exact field. No new physics is claimed.

Keywords: Liénard–Wiechert fields, retarded time, causality-preserving interpolation, Bernstein polynomials, double-float arithmetic, stereographic compactification, synchrotron radiation, WebGL2

1 Introduction and relationship to prior work

The field of a moving point charge is given in closed form by the Liénard–Wiechert expressions evaluated at the retarded time [1]. Computing it numerically at many observation points is routine in accelerator physics and in teaching. Existing tools take four routes.

1. **Grid solvers (FDTD/PIC)** such as WarpX [2] evolve $(\mathbf{E}, \mathbf{B})$ on a mesh. They handle self-consistent dynamics but need resolution of order $R_{\text{orb}}\gamma^{-3}$ for a synchrotron pulse. Radiation diagnostics inside PIC codes, such as RaDiO [3], deposit each particle's LW field forward onto a detector in space and time.

2. **Wavefront methods.** Shintake's wavefront method, with a real-time 2D animation of synchrotron and undulator radiation [4, 5], and LANL's CoSyR [6] build fields from wavefronts emitted along stored trajectories (scatter).

3. **Per-observer retarded-time solves.** PyCharge [7] and the accompanying visualization work [8] solve the retarded-time equation at each observation point for analytic 3D trajectories, as batch computations (PyCharge 2 uses JAX and can run on GPUs). The Liénard–Wiechert solver of Ryne *et al.* [9] locates the retarded time on a stored particle history by bisection over the stored values, refines it with Brent's method, and uses double-double arithmetic where round-off matters. Zhao *et al.* [10] bracket and bisect on a stored history and then solve a quadratic on a linear segment. Nakayama, Oda and Yasuda [11] scan a stored worldline for its crossing of an observer's past light cone and solve a quadratic on the linearly interpolated segment, in an interactive teaching visualization. Frequency-domain codes such as SRW [12] compute the radiation in the near field by integrating over emission time.

4. **Coordinate compactification.** Maps such as $R = L\tan(\theta/2)$ are standard in numerical relativity for mapping unbounded domains to finite grids [13]. Related algebraic maps of the half-line are standard in spectral methods [14].

Gather versus scatter. Here each observation point gathers: it asks each source which event on its stored worldline lies on its past light cone. The split between per-observer "retarded-time" and per-source "source-time-dominant" formulations is long established for moving-source noise prediction in aeroacoustics [15, 16]. Bisection on a stored history [9, 10] and per-point retarded-time solves [7] are also established. Ryne *et al.* [9] solve the same root problem by bisection on a stored history; relative to that work, the additions here are a proof that the root is unique for the interpolated curve itself (not only for the true motion), compactified storage of the history, and the per-pixel validity indicator. What this work contributes is an engineering combination: a bounded-cost, single-precision GPU kernel with a guaranteed unique root, a compact radial history, a per-pixel validity indicator, and measured, reproducible accuracy.

Contributions

- A closed-form, branch-light causality certificate for cubic Hermite worldline segments based on the Bernstein coefficients of $\|\mathbf{z}'\|^2$ (Proposition 1), and its degree-8 extension to the quintic segments used for exactly known paths (Section 2.7). The cubic certificate is never stricter than the control-vector limiter of version 5 (Theorem 1). On random segments the version-5 limiter allows a median 0.26 of the largest safe blend, the new certificate 0.46.

- An account of where single precision loses accuracy (Sections 2.4 and 2.6). For the version-5 algorithm it is one subtraction in the root function and absolute knot storage; double-float arithmetic at the knots removes most of that. The error that remains in version 6 at $\gamma \gtrsim 70$ comes from the float32 offset inside the segment. A cancellation-free incremental root function, with the chord deficit from the CPU in float64, removes it. On Apple's Metal compiler every double-float rounding step must be guarded against reassociation.

- An open, single-file browser tool and TypeScript library with a built-in verification suite, an independent cross-check against PyCharge, and a shader-side validity indicator.

2 Formulation

2.1 Retarded Liénard–Wiechert field and a cancellation-free kernel

For a point charge q with trajectory $\mathbf{z}(t_{\text{ret}})$, $\boldsymbol{\beta} = \dot{\mathbf{z}}/c$ and $\dot{\boldsymbol{\beta}} = \ddot{\mathbf{z}}/c$, let $R = c(t - t_{\text{ret}})$ be the retarded radius at observation time t, so that $d\mathbf{z}/dR = -\boldsymbol{\beta}$. The retarded radius R_{ret} for an observer at $\mathbf{x}$

solves

$$f(R) = R - \|\mathbf{x} - \mathbf{z}(R)\| = 0, \qquad \hat{\mathbf{n}} = \frac{\mathbf{x} - \mathbf{z}(R_{\text{ret}})}{\|\mathbf{x} - \mathbf{z}(R_{\text{ret}})\|}, \tag{1}$$

and the field is [1]

$$\mathbf{E}(\mathbf{x}, t) = \frac{q(1-\beta^2)}{\kappa^3 R^2}(\hat{\mathbf{n}} - \boldsymbol{\beta}) + \frac{q}{c\,\kappa^3 R}\hat{\mathbf{n}} \times \left[(\hat{\mathbf{n}} - \boldsymbol{\beta}) \times \dot{\boldsymbol{\beta}}\right], \qquad \kappa = 1 - \hat{\mathbf{n}} \cdot \boldsymbol{\beta}, \qquad \mathbf{B} = \hat{\mathbf{n}} \times \mathbf{E}. \tag{2}$$

Charges move in the x–y plane and the field is evaluated in that plane, where $\mathbf{E}$ is in-plane and $\mathbf{B}$ is along z. An optional display softening $R^2 \to R^2 + \epsilon^2$ is off ($\epsilon = 0$) for all accuracy figures.

To avoid cancellation in $1 - \hat{\mathbf{n}} \cdot \boldsymbol{\beta}$ and $1 - \beta^2$ as $\beta \to 1$, the history stores the proper velocity $\mathbf{u} = \gamma\boldsymbol{\beta}$, recovers $1 - \beta^2 = (1 + \|\mathbf{u}\|^2)^{-1}$, and evaluates

$$\kappa = \frac{1-\beta^2}{1+\beta} + \frac{1}{2}\beta\,\|\hat{\mathbf{n}} - \hat{\boldsymbol{\beta}}\|^2, \qquad \hat{\mathbf{n}} - \boldsymbol{\beta} = \frac{1-\beta^2}{1+\beta}\,\hat{\mathbf{n}} + \beta\,(\hat{\mathbf{n}} - \hat{\boldsymbol{\beta}}). \tag{3}$$

The radiation term is evaluated as

$$\hat{\mathbf{n}} \times \left[(\hat{\mathbf{n}} - \boldsymbol{\beta}) \times \dot{\boldsymbol{\beta}}\right] = (\hat{\mathbf{n}} - \boldsymbol{\beta})(\hat{\mathbf{n}} \cdot \dot{\boldsymbol{\beta}}) - \kappa\,\dot{\boldsymbol{\beta}}, \tag{4}$$

which follows from $\hat{\mathbf{n}} \cdot (\hat{\mathbf{n}} - \boldsymbol{\beta}) = \kappa$, with κ and $\hat{\mathbf{n}} - \boldsymbol{\beta}$ from Eq. (3) and $\hat{\mathbf{n}}$ from the separation $\mathbf{x} - \mathbf{z}$ formed in double-float arithmetic (Section 2.4). (The cross-product form of v6.0.0 rebuilds $\kappa \sim 1/(2\gamma^2)$ from $O(1)$ float32 terms.) With Eq. (4) and the Newton fix of Section 2.5, the version-6 pulse-sweep error at $M_T = 512$ fell from 0.0095% to 0.0077% at $\gamma = 22.4$ and from 0.069% to 0.036% at $\gamma = 70.7$.

2.2 Stereographic radial shells

Each source's history is resampled onto M_T cell-centred shells $\theta_m = (m + \frac{1}{2})\Delta\theta$, $\Delta\theta = \pi/M_T$, at

$$R(\theta) = L \tan\left(\frac{\theta}{2}\right), \qquad \Delta R(\theta) = \frac{L^2 + R^2}{2L}\,\Delta\theta. \tag{5}$$

Spacing is nearly uniform for $R \lesssim L$, where radiation forms, and grows as R^2 beyond. The outermost shell is at $L \tan(\frac{\pi}{2} - \frac{\Delta\theta}{4}) \approx 4LM_T/\pi$. The map is one of the algebraic maps of the half-line onto a finite interval used in spectral methods; Boyd's rational Chebyshev functions on $[0, \infty)$ use the related map $y = L\cot^2(t/2)$ [14]. It provides reach within a fixed texture, not uniform resolution to infinity: accuracy falls once ΔR no longer resolves the motion (Section 4.3). Shells are sampled from an append-only ring buffer in retarded time, so the textures are a resampled view of that buffer, refreshed each step. Here the buffer holds about 260 length units of history; shells beyond that depth are flagged (Section 2.8). For preset motions the shells can instead hold exact samples of the path (Section 2.7).

2.3 Causality certificates for cubic Hermite segments

On a segment $[R_1, R_2]$, $\Delta R = R_2 - R_1$, with knot positions $\mathbf{z}_1, \mathbf{z}_2$ and knot velocities $\boldsymbol{\beta}_1, \boldsymbol{\beta}_2$ ($\|\boldsymbol{\beta}_i\| \le \beta_{\max} < 1$), write $\mathbf{v} = (\mathbf{z}_2 - \mathbf{z}_1)/\Delta R$ and assume the chord is subluminal, $\|\mathbf{v}\| < \beta_{\max}$. Consider a cubic Hermite curve with endpoint R-derivatives blended toward the secant by $\alpha \in [0, 1]$,

$$\mathbf{V}_0(\alpha) = (1-\alpha)\mathbf{v} - \alpha\boldsymbol{\beta}_1, \qquad \mathbf{V}_2(\alpha) = (1-\alpha)\mathbf{v} - \alpha\boldsymbol{\beta}_2, \qquad \mathbf{V}_1(\alpha) = \mathbf{v} + \alpha(2\mathbf{v} + \boldsymbol{\beta}_1 + \boldsymbol{\beta}_2). \tag{6}$$

It interpolates the knots for every α, and its derivative is exactly the quadratic Bernstein form $\mathbf{z}'(s) = \sum_{k=0}^{2} B_{k,2}(s)\mathbf{V}_k(\alpha)$, $s = (R - R_1)/\Delta R$.

Theorem 1 (Control-vector certificate, version 5)**.** *$\mathbf{V}_0$ and $\mathbf{V}_2$ lie in the ball $\|\cdot\| \le \beta_{\max}$ for all α by convexity. If α^* is the largest $\alpha \in [0, 1]$ with $\|\mathbf{V}_1(\alpha)\| \le \beta_{\max}$ (a closed-form quadratic root), then $\|\mathbf{z}'(R)\| \le \beta_{\max}$ on the segment by the convex-hull property of Bernstein polynomials.*

The bound is a known technique: velocity limits through Bernstein control points are standard in CNC and robot trajectory planning [17, 18], and blending tangents toward the secant is a vector form of monotonicity-preserving Hermite interpolation [19]. Applied to retarded time it is useful but conservative. For a circular arc of turning angle φ, $\|\mathbf{V}_1\| = \beta(1 + \varphi^2/8 + O(\varphi^4))$, so the limiter fires on smooth arcs whose true speed never exceeds β. On the version-5 benchmark it fired on 4–87% of segments that were never superluminal.

Proposition 1 (Squared-speed certificate). $\|\mathbf{z}'(s)\|^2 = \sum_{k=0}^{4} B_{k,4}(s)\, c_k$ *with*

$$c_0 = \mathbf{V}_0 \cdot \mathbf{V}_0, \quad c_1 = \mathbf{V}_0 \cdot \mathbf{V}_1, \quad c_2 = \tfrac{1}{3}\left(2\,\mathbf{V}_1 \cdot \mathbf{V}_1 + \mathbf{V}_0 \cdot \mathbf{V}_2\right), \quad c_3 = \mathbf{V}_1 \cdot \mathbf{V}_2, \quad c_4 = \mathbf{V}_2 \cdot \mathbf{V}_2.$$

Each $c_k(\alpha)$ is quadratic in α, and $c_k(0) = \|\mathbf{v}\|^2 < \beta_{\max}^2$. Let α_{sq} be the smallest $\alpha \in (0, 1]$ at which some $c_k(\alpha) = \beta_{\max}^2$ (or 1 if none); if $\|\mathbf{v}\| = \beta_{\max}$, set $\alpha_{sq} = 0$. Then for every $\alpha \le \alpha_{sq}$, $\|\mathbf{z}'\| \le \beta_{\max}$ on the segment, and $\alpha_{sq} \ge \alpha^*$.

Proof. The product of two quadratic Bernstein expansions is a quartic Bernstein expansion with the stated coefficients; since $B_{k,4} \ge 0$ and $\sum_k B_{k,4} = 1$, $\|\mathbf{z}'\|^2 \le \max_k c_k \le \beta_{\max}^2$. c_0 and c_4 never exceed $\beta_{\max}^2$ by Theorem 1, and by continuity no c_k exceeds it before its first crossing. If all $\|\mathbf{V}_k\| \le \beta_{\max}$ then by Cauchy–Schwarz every $c_k \le \beta_{\max}^2$, so no c_k exceeds $\beta_{\max}^2$ on $[0, \alpha^*]$; outside degenerate tangential contact this gives $\alpha_{\mathrm{sq}} \ge \alpha^*$. □

Computing α_{sq} costs three quadratic solves. The guarantee holds in exact arithmetic. In a float32 emulation it held for $\beta \le 0.99999$ with $\beta_{\max} = 0.9999995$, but it was violated by up to $\sim 1 - \beta_{\max}$ at $\beta = 0.999999$, so $\beta_{\max} - \beta$ must exceed a few float32 ulps. With the speed bound on every segment, $f'(R) = 1 + \hat{\mathbf{n}} \cdot \mathbf{z}'(R) \ge 1 - \beta_{\max} > 0$, so f is strictly increasing on the stored domain and each observer has exactly one retarded root there. The limited worldline is C^0 everywhere and C^1 at a knot when neither neighbouring segment is limited; where a limiter acts, the velocity direction can be discontinuous at the knot. A globally C^1 subluminal cubic does not always exist (for alternating chords $\pm c$ with free knot slopes it requires $c \le 2\beta/3$), whereas raising the degree restores it. Section 2.7 uses quintic segments where the path is known exactly.

2.4 Single-precision accuracy and its cause

The version-5 algorithm. We attributed the error of the version-5 float32 algorithm, in a CPU emulation, by promoting one stage at a time to float64 [23]. At $\gamma = 70.7$ ($M_T = 512$), rounding only the root-function arithmetic $\mathbf{x} - \mathbf{z}$, $\|\cdot\|$ and $R - \|\cdot\|$ to float32 produces 12.4% error, and storing knot positions in absolute float32 coordinates produces 2.4%. Every other stage contributes at most 0.05%. The cause is conditioning: near the pulse crest $f'(R) \approx \kappa \sim \gamma^{-2}$, so an absolute error δ in f moves the retarded point by δ/κ, and float32 positions near $|\mathbf{x}| \sim 1$ carry $\delta \sim 6 \times 10^{-8}$.

Version 6: double-float arithmetic at the knots. Knot positions are stored as pairs $\mathbf{z} = \mathbf{z}_{\mathrm{hi}} + \mathbf{z}_{\mathrm{lo}}$ of float32 values. Chords are formed from both parts, and $\mathbf{x} - \mathbf{z}_1$, its length and $R_1 - \|\mathbf{x} - \mathbf{z}_1\|$ are evaluated with error-free transformations (double-float arithmetic [20, 21]). This lowers the version-5 error about 100-fold. Error-free transformations only work if the compiler evaluates them as written. Apple's Metal shader compiler (reached through ANGLE) does not. On an M1 Max, an earlier version guarded by multiplying by a uniform equal to 1 fell back to float32 accuracy, because sums such as $(a - x) + (b - y)$ were regrouped as $(a + b) - (x + y)$. The shader therefore routes every rounding step through an identity the compiler cannot see through (reinterpreting the float as an integer and XOR-ing it with a uniform equal to 0), and splits operands for the exact products by masking mantissa bits. A probe page in the repository tests each building block on the user's GPU against exact results.

What limits version 6 (corrected in v6.2). Version 6.1 stated that the remaining float32 error came from the root function and that everything other than the knot-level root arithmetic could stay in float32. That is not what limits version 6 at high γ. In the version-6 kernel, the offset inside the segment, $\Delta\mathbf{z}(s) = \mathbf{z}(R) - \mathbf{z}_1$, and $\Delta R = R - R_1$ are float32 Hermite evaluations that are added to the double-float separation. $\Delta\mathbf{z}$ therefore carries an absolute error of about $\varepsilon\,\|\Delta\mathbf{z}\|$, where $\varepsilon = 2^{-24}$. The part of f that changes across the segment is

$$\Delta R - \|\Delta\mathbf{z}\| = \Delta R\,\frac{1 - |\bar{\mathbf{v}}|^2}{1 + |\bar{\mathbf{v}}|}, \qquad \bar{\mathbf{v}} = \Delta\mathbf{z}/\Delta R,$$

and the chord deficit $1 - |\bar{\mathbf{v}}|^2$ is of order γ^{-2} near the crest. Its float32 relative error, and hence the relative error of the root increment, is of order $\varepsilon\gamma^2$. Double-float arithmetic at the knots does not remove this term. The evidence is direct. A diagnostic build of the new kernel (Section 2.6, before its direction fix) that forms only the chord deficit naively in float32, as $1-|\Delta\mathbf{z}/\Delta R|^2$, brings v6f back to the version-6 error level, within a factor of 2, in all 18 cases tested (e.g. 1.05% for both at $\gamma = 224$, $M_T = 512$, known path) [23]. The app's suite shows the same from the other side. At $\gamma = 70.7$, the GPU error of both v6f kernels agrees within a few per cent with that of the same spline evaluated in float64 (v6f 0.0233%, 0.00588%, 0.00176%; v6f precise 0.0236%, 0.00607%, 0.00160%; spline 0.0236%, 0.00602%, 0.00166% at $M_T = 512$, 1024, 2048), while version 6 gives 0.0363%, 0.0158% and 0.00703% (`results/suite_swiftshader_v6_2.tsv`). In Table 2 this floor dominates version 6 at $\gamma = 224$, $M_T \geq 512$, and $\gamma = 70.7$, $M_T = 2048$; it is part of the error at $\gamma = 70.7$, $M_T = 512$.

Two further float32 effects above $\gamma \approx 100$ were corrected in version 6. (i) The chord-speed clamp that guards against superluminal input compared float32 differences of adjacent knots. At $M_T = 2048$ their relative rounding ($\sim 3\times10^{-5}$) exceeds $1-\beta$, so physical knots were wrongly moved. The clamp now uses the full high/low positions in double precision. (ii) The Newton derivative was floored at 5×10^{-5}, larger than the true f' at $\gamma = 224$. The floor is now $\frac{1}{2}(1 - \beta_{\mathrm{max}})$, which the certificate guarantees (in exact arithmetic) is below f'.

How much accuracy is available. At $\gamma = 70.7$ the swept pulse is about 15 float32 ulps wide in observer position, and rounding the observer coordinates alone shifts the exact field by 3.2% (Table 2). A float32-addressed method can therefore only be judged against the exact field at the rounded point, which is the metric used here, or with observers placed by double-float coordinates. The library's readback API supports these (the rendered view uses float32 pixel coordinates), and they are required above $\gamma \approx 100$.

2.5 Root search (version-6 kernel)

Because f is monotone at the stored knots (subluminal chords), the solver first bisects over knot indices, evaluating f at knots only (three texel fetches each, no spline evaluation). It then runs safeguarded Newton inside the bracketing segment on the certified cubic, iterating to convergence (at most 12 steps). The segment index and the fraction within it are kept as separate values, so float32 resolution of a combined coordinate never limits the retarded point. Since v6.0.0 the safeguard keeps a closed bracket $[s_{\mathrm{lo}}, s_{\mathrm{hi}}]$, accepts a Newton step that lands on either end, falls back to bisection otherwise, and stops when $f = 0$ exactly; in v6.0.0 a step that reached the root exactly could be bisected away from it. The same fix is in the JAX port. For on-screen rendering only, a display path reuses the per-segment coefficients across Newton steps, bisects over knots in float32 with one double-float bracket check, and forms $f = (R^2 - |\mathbf{d}|^2)/(R + |\mathbf{d}|)$ without a double-float square root. This takes about a third of the operations of the full path, and rendered output differs from the full solve by at most 3/255 per channel. Readback, including every table here, always takes the full path.

2.6 The v6f kernel: float32 incremental root function

v6.2.0 has four kernels. v6f is the default in the app and in the library. *v6f precise* (`v6fp`) is v6f plus the field-floor fix described below. v6 and v5 remain selectable; the app's kernel button cycles v6f, v6f precise, v6, v5. The v6 program is bit-identical to the version-6 kernel on main before this release (commit `904bcce`, with Eq. (4) and the Newton fix; reference copies in `reference/`). In the library, `PRODUCTION_SHADERS` now means v6f, `V6FP_SHADERS` selects v6f precise and `V6_SHADERS` this version-6 kernel. Both v6f kernels are the version-6 shader source compiled with preprocessor switches: `F32N` and `F32N_NOFIX` for v6f, `F32N` alone for v6f precise. They keep double-float arithmetic only for $f_1 = R_1 - \|\mathbf{d}_1\|$ and $\mathbf{d}_1 = \mathbf{x} - \mathbf{z}_1$ at the bracketing knot.

Incremental root function. Let knot 1 be the bracketing knot. Because R increases into the past, knot 1 is the later event. Write $\Delta\mathbf{z} = \mathbf{z}(R) - \mathbf{z}_1$, $\ell = \|\Delta\mathbf{z}\|$, $\rho = R - R_1 = s\,\Delta R$, $L_1 = \|\mathbf{d}_1\|$ and $L = \|\mathbf{d}_1 - \Delta\mathbf{z}\| = \|\mathbf{x} - \mathbf{z}(R)\|$. Then

$$f(R) = f_1 + \underbrace{[\rho - \ell]}_{\geq 0} + \underbrace{[\ell + L_1 - L]}_{\geq 0}, \tag{7}$$

where the first bracket is non-negative because the segment is subluminal and the second by the triangle inequality. Both are formed without cancellation:

$$\rho - \ell = \frac{\rho\, D}{1 + \ell/\rho}, \qquad D = 1 - |\bar{\mathbf{v}}|^2, \quad \bar{\mathbf{v}} = \Delta\mathbf{z}/\rho, \tag{8}$$

$$\ell + L_1 - L = \begin{cases} \dfrac{2\,|\mathbf{d}_1 \times \Delta\mathbf{z}|^2}{(\ell L_1 - \mathbf{d}_1 \cdot \Delta\mathbf{z})\,(\ell + L_1 + L)}, & \mathbf{d}_1 \cdot \Delta\mathbf{z} \leq 0, \\ \dfrac{2\,(\ell L_1 + \mathbf{d}_1 \cdot \Delta\mathbf{z})}{\ell + L_1 + L}, & \mathbf{d}_1 \cdot \Delta\mathbf{z} > 0. \end{cases} \tag{9}$$

The second line follows from $(\ell + L_1)^2 - L^2 = 2(\ell L_1 + \mathbf{d}_1 \cdot \Delta\mathbf{z})$, and the first from $(\ell L_1)^2 - (\mathbf{d}_1 \cdot \Delta\mathbf{z})^2 = |\mathbf{d}_1 \times \Delta\mathbf{z}|^2$. The case split depends on the time direction. At the pulse crest the charge moves toward the observer, so the earlier point $\mathbf{z}(R)$ is farther away. $\Delta\mathbf{z}$ is then nearly antiparallel to $\mathbf{d}_1$, and $\ell L_1 + \mathbf{d}_1 \cdot \Delta\mathbf{z}$ cancels. The well-conditioned denominator is $\ell L_1 - \mathbf{d}_1 \cdot \Delta\mathbf{z} \approx 2\ell L_1$. A form written for a parameter running forward in time has its cancelling sum in the denominator in this convention; that was corrected in testing. No term of Eqs. (7)–(9), or of D below (where $\lambda_2 < 0$ makes some $O(\gamma^{-2})$ terms negative), subtracts quantities of order 1, so reassociation cannot cancel them badly and these steps carry no compiler barrier.

Chord deficit. D must not be formed as $1 - |\bar{\mathbf{v}}|^2$ from a float32 $\bar{\mathbf{v}}$, which is exactly the version-6 floor. $\bar{\mathbf{v}}(s)$ is the mean of the hodograph over $[0, s]$, $\bar{\mathbf{v}} = \sum_k I_k(s)\mathbf{V}_k$ with $I_k(s) = s^{-1}\int_0^s B_{k,2}$. Substituting Eq. (6) writes it as an affine combination of the chord velocity $\mathbf{P}_v = \mathbf{v}$ and the knot points $\mathbf{P}_1 = -\boldsymbol{\beta}_1$, $\mathbf{P}_2 = -\boldsymbol{\beta}_2$:

$$\bar{\mathbf{v}}(s) = \lambda_v \mathbf{P}_v + \lambda_1 \mathbf{P}_1 + \lambda_2 \mathbf{P}_2, \qquad \lambda_1 = \alpha(1-s)^2, \quad \lambda_2 = -\alpha s(1-s), \quad \lambda_v = 1 - \lambda_1 - \lambda_2, \tag{10}$$

exactly, for any blend α. (In the Bernstein indexing of Eq. (6), $\lambda_1 = \alpha(I_0 - I_1)$ and $\lambda_2 = \alpha(I_2 - I_1)$, with $I_0 - I_1 = (1-s)^2$ and $I_2 - I_1 = -s(1-s)$.) For weights with $\sum_i \lambda_i = 1$, of any sign,

$$1 - \Big|\sum_i \lambda_i \mathbf{P}_i\Big|^2 = \sum_i \lambda_i D_i + \sum_{i<j} \lambda_i \lambda_j\, |\mathbf{P}_i - \mathbf{P}_j|^2, \qquad D_i = 1 - |\mathbf{P}_i|^2. \tag{11}$$

The CPU computes $D_v = 1 - |\mathbf{v}|^2$ in float64 from the high/low knot positions, with the shader's chord clamp, and stores it per knot in a texture channel that was unused. The knot deficits are $D_{1,2} = (1 + |\mathbf{u}_{1,2}|^2)^{-1}$ from the stored proper velocity. The pair terms $|\mathbf{v} + \boldsymbol{\beta}_1|^2$, $|\mathbf{v} + \boldsymbol{\beta}_2|^2$ and $|\boldsymbol{\beta}_1 - \boldsymbol{\beta}_2|^2$ are squared lengths of small differences, formed once per segment. No term subtracts

two quantities close to 1. For the quintic segments of Section 2.7 the hodograph is quartic with control points $\mathbf{Q}_0, \ldots, \mathbf{Q}_4$. Because $\sum_k \mathbf{Q}_k = 5\mathbf{v}$, $\mathbf{Q}_2$ can be eliminated, giving an affine combination of $(\mathbf{v}, \mathbf{Q}_0, \mathbf{Q}_1, \mathbf{Q}_3, \mathbf{Q}_4)$ with $\lambda_k = \alpha_q(I_k - I_2)$, $I_k = s^{-1}\int_0^s B_{k,4}$. Here $\mathbf{Q}_0 = -\boldsymbol{\beta}_1$, $\mathbf{Q}_4 = -\boldsymbol{\beta}_2$, $\mathbf{Q}_1 = \mathbf{Q}_0 + \mathbf{w}_1$ and $\mathbf{Q}_3 = \mathbf{Q}_4 - \mathbf{w}_2$, with $\mathbf{w}_k = \frac{1}{4}\Delta R\,\mathbf{a}_k$ and $\mathbf{a}_k = \mathbf{z}''(R_k)$ the stored acceleration. So $D_{Q_1} = D_1 + 2\boldsymbol{\beta}_1 \cdot \mathbf{w}_1 - |\mathbf{w}_1|^2$ and $D_{Q_3} = D_2 - 2\boldsymbol{\beta}_2 \cdot \mathbf{w}_2 - |\mathbf{w}_2|^2$. Segments limited by the certificate ($\alpha_q < 1$) are covered by the same form.

Knot bisection in float32 with a double-float bracket check. The knot bisection uses the float32 function $g_m = R_m - \|\mathbf{x}_{\text{hi}} - \mathbf{z}_{\text{hi},m}\|$, which differs from the true $f_m = f(R_m)$ by at most a rounding bound δ. The knot values are strictly increasing, $f_{m+1} - f_m \geq \Delta R_m(1 - \|\mathbf{v}_m\|) > 0$. A sign of g can therefore be wrong only at knots with $|f_m| \leq \delta$, which by monotonicity form one contiguous run next to the root. Every decision outside that run is correct, so the float32 bracket ends inside or next to it. The kernel then evaluates f in double-float at both bracket knots and slides the one-segment bracket by one knot at a time (at most 16 steps) until $f_{\text{lo}} < 0 \leq f_{\text{hi}}$ holds in double-float. Because f is monotone along the knots, this is the bracket a double-float bisection finds; the float32 pass only changes how many double-float evaluations are needed. In 18 test cases the output was identical to double-float bisection, with 2.0–2.7 double-float and 11–13 float32 knot evaluations per pixel and source, against 11–13 double-float knot evaluations for version 6 [23].

Direction rounding and the field-floor fix (v6f precise). With the chord deficit fixed, v6f on exactly sampled paths is limited at about $0.6 \times 3\varepsilon\gamma$ (about 7×10^{-6} relative at $\gamma = 70.7$). Promoting stages one at a time to float64, in an emulation that matches the GPU, locates this floor in the float32 rounding of directions, not in the κ formula ($\leq 2 \times 10^{-7}$) or the $1/R$ factors [23]. Near the crest $\hat{\mathbf{n}}$, $\hat{\boldsymbol{\beta}}$ and the chord are parallel to within about $1/\gamma$. A rotation by ε of any one of them is then a relative error of about $\varepsilon\gamma$ in $1 - \hat{\mathbf{n}} \cdot \hat{\boldsymbol{\beta}}$, in $\hat{\mathbf{n}} \cdot \mathbf{a}$ and in the root's $\mathbf{d}_1 \times \Delta\mathbf{z}$. Five or six stages contribute comparable amounts. The fix takes every cancelling cross or dot product against the double-float separation $\mathbf{d}_1 = \mathbf{d}_{\text{hi}} + \mathbf{d}_{\text{lo}}$, once per pixel and source, with exact two-product multiplications. The segment is linear in its data $(\mathbf{v}, \boldsymbol{\beta}_1, \boldsymbol{\beta}_2, \mathbf{a}_1, \mathbf{a}_2)$, so the scalars $\psi_k = \mathbf{d}_1 \times \mathbf{V}_k$ (hodograph control points) and $\chi_k = \mathbf{d}_1 \times \mathbf{C}_k$ (position control points relative to $\mathbf{z}_1$) give

$$\mathbf{d}_1 \times \mathbf{z}'(s) = \sum_k B_k(s)\,\psi_k, \qquad \mathbf{d}_1 \times \Delta\mathbf{z}(s) = \sum_k B_k(s)\,\chi_k, \tag{12}$$

in the Bernstein (or, for the cubic position, Hermite) basis of the segment, with no rounding of a direction. The second sum is used in Eq. (9). At the root, with $\hat{\boldsymbol{\beta}} = -\mathbf{z}'/|\mathbf{z}'|$, $\hat{\boldsymbol{\beta}}_\perp$ the in-plane perpendicular and ϕ the angle from $\hat{\boldsymbol{\beta}}$ to $\hat{\mathbf{n}}$,

$$\sin\phi = \frac{\mathbf{d}_1 \times \mathbf{z}' - \Delta\mathbf{z} \times \mathbf{z}'}{|\mathbf{z}'|\,L}, \quad 1 - \cos\phi = \frac{\sin^2\phi}{1 + \cos\phi}, \quad \hat{\mathbf{n}} - \hat{\boldsymbol{\beta}} = \sin\phi\,\hat{\boldsymbol{\beta}}_\perp - (1 - \cos\phi)\,\hat{\boldsymbol{\beta}}, \tag{13}$$

and then $\kappa = (1 - \beta) + \beta(1 - \cos\phi)$ and $\hat{\mathbf{n}} - \boldsymbol{\beta} = (1 - \beta)\hat{\mathbf{n}} + \beta(\hat{\mathbf{n}} - \hat{\boldsymbol{\beta}})$, as in Eq. (3). Likewise $\hat{\mathbf{n}} \cdot \mathbf{a}(s) = [\sum_k w_k(s)(\mathbf{d}_1 \cdot \mathbf{a}_k) - \Delta\mathbf{z} \cdot \mathbf{a}(s)]/L$, with w_k the interpolation weights. $\mathbf{d}_1 \cdot \mathbf{a}_k$ uses exact products at the two inner knots and plain float32 at the outer Catmull–Rom knots, as do the quintic's $\mathbf{w}_k$ terms. The fix is the only difference between the two v6f kernels: v6f precise compiles it in, and the default v6f leaves it out (`F32N_NOFIX`) because of its cost on the M1 (below). It runs only when $|\mathbf{u}| > 4$ at a knot of the segment ($\gamma > 4.1$), since below that the floor is under $\sim 3 \times 10^{-7}$. On the known path it lowers the error 2–17× (4–17× in Table 1), to about 1.2×10^{-6} relative at $\gamma = 70.7$ and $4\text{–}6 \times 10^{-6}$ at $\gamma = 224$ for $M_T \geq 512$. That is close to the floor set by float32 storage of $\mathbf{u}$ and $\mathbf{a}$. On ring-buffer and live histories, limited by interpolation, the pulse sweep changes by $0.9\text{–}1.2\times$.

Cost. On an Apple M1 Max (Safari, WebGL2 on Metal), in the app's default scene with 32 sources on the known path, v6f renders at 42 fps and version 6 at 43 fps (Table 5; `results/suite_m1max_v6_2.txt`). v6f's advantage in SwiftShader, about $0.5\times$ ($0.49\text{–}0.52\times$) the frame time of version 6,

did *not* carry over to the M1, where the two run at the same speed. The field-floor fix costs about 23% of the frame rate there: v6f precise renders at 33 fps, $0.77\times$ version 6, about twice the fix's SwiftShader cost. This is why the default leaves the fix out. The SwiftShader figures are medians of five interleaved rounds (default scene, 32 sources, $M_T = 128$; `results_frame_ratio_v62.json` in `validation/f32newton/f32newton_floor/`): v6f precise $0.55\times$ the frame time of version 6 with ring-buffer histories and $0.58\times$ on the known path, at $\gamma = 1.67$ and 22.4 alike, with the fix adding 11–12% even below its gate, because SwiftShader evaluates the gated branch. A hand count times measured evaluation counts (`validation/f32newton/f32newton_2d/opcount_2d.py`) gives 0.76–$0.81\times$ the arithmetic of version 6 on the display path without the fix and 1.01–$1.14\times$ with it at $\gamma > 4.1$ (readback path 0.23–$0.42\times$). Neither the counts nor SwiftShader predicted the M1 timings. Version 6 at commit `06edf87` rendered at 62 fps in Chrome on the same machine (`results/suite_m1max_web_kform.txt`); browser and code differ, so that figure is not compared here.

		known path (exact knots)			live ring buffer		
γ	M_T	v6	v6f	v6f precise	v6	v6f	v6f precise
70.7	128	0.12%	0.0014%	8.5×10^{-5}%	—	—	—
70.7	512	0.027%	6.9×10^{-4}%	1.4×10^{-4}%	0.043%	0.024%	0.024%
70.7	2048	0.0078%	8.5×10^{-4}%	1.2×10^{-4}%	0.012%	0.0017%	0.0015%
224	128	4.4%	0.018%	0.0035%	—	—	—
224	512	1.1%	0.0021%	5.5×10^{-4}%	0.68%	0.067%	0.068%
224	2048	0.28%	0.0026%	4.0×10^{-4}%	0.31%	0.0048%	0.0052%

Table 1: Relative L_2 error on the 256-point pulse sweep (one synchrotron source, no softening, double-float observers), software rendering. "Known path": shells hold exact samples joined by quintic segments (Section 2.7); library readback, `results/table2_v6_2.tsv`. "Live": the app's ring-buffer history after 600 live steps (`validation/f32newton/f32newton_floor/`, `acc_before.log` for v6f and `acc_final.log` for v6f precise). On the known path the version-6 error is the chord-deficit floor, v6f is limited by direction rounding, and v6f precise reaches the float32 storage floor. On live histories both v6f kernels are limited by interpolation of the history.

2.7 Histories: dragged sources and known paths

Physical dragging. A dragged charge in the app follows the pointer as an intent, on a worldline whose position, velocity and acceleration are consistent. A two-loop controller limits closing speed by braking distance, acceleration to $1.25\sqrt{M_T/128}$ per time unit, the jerk, and the speed to a cap below c. The stored acceleration equals the applied change of velocity, and positions follow the trapezoid rule. On release the charge returns to its preset path through the same dynamics, and control passes back only when the two worldlines coincide.

Known-path mode. For presets whose position, velocity, acceleration and jerk are known in closed form, the shells can hold exact samples of the path. A segment between two exact knots is a quintic Hermite curve matching position, velocity and acceleration at both knots. Its hodograph is quartic with control points $\mathbf{Q}_k$, so $\|\mathbf{z}'\|^2$ has degree-8 Bernstein coefficients $c_k = \sum_{i+j=k} \binom{4}{i}\binom{4}{j}\, \mathbf{Q}_i \cdot \mathbf{Q}_j / \binom{8}{k}$. With $\mathbf{Q}_k(\alpha) = \mathbf{v} + \alpha(\mathbf{Q}_k - \mathbf{v})$ each c_k is quadratic in α, and the certificate of Proposition 1 carries over. Unlimited segments are C^2; a limited segment no longer matches the endpoint derivatives and is C^0 there, as the cubic is. Knots in a dragged stretch, and purely live sources, use the ring buffer. Through a drag, return and hand-off on the orbit preset ($\beta = 0.8$, 4 sources, $M_T = 512$), the known-path and ring-buffer fields differ by at most 3.3×10^{-6} RMS. An *Auto* shell count picks M_T from the view's largest retarded radius (128 for the default view, 512 above $\gamma = 100$). With the version-6 kernel, known-path mode matched the ring buffer's $M_T = 2048$ accuracy with 64$\times$ fewer shells over the default view at $\gamma \le 71$, and 4$\times$ fewer at $6 < R < 20$ (not re-measured for v6f).

2.8 Validity indicator

For each source the shader computes, at the retarded point,

$$q = \max\left(\frac{\Delta R\, \gamma\, |\dot{\beta}_\perp|}{c\, \max(\beta^2, 0.04)},\ \frac{6\, \|\Delta \mathbf{u}\|}{\bar{u} + 0.1},\ q_{\text{acc}} \right),$$

$$q_{\text{acc}} = 7.5\, \frac{|\hat{\mathbf{n}} - \boldsymbol{\beta}| + \kappa}{\kappa^3 R}\, \frac{\max\left(\|\Delta^2 \mathbf{a}\|,\ \|(\boldsymbol{\beta}_2 - \boldsymbol{\beta}_1)/\Delta R + \tfrac{1}{2}(\mathbf{a}_1 + \mathbf{a}_2)\| \right)}{\|\mathbf{E}_{\text{source}}\|}.$$

The first term is shell spacing times γ over the local radius of curvature, with β^2 floored at 0.04 ($\beta = 0.2$) so that charges at rest are not flagged. The second is the relative change of proper velocity across the segment. The third, added for dragged histories, converts two acceleration errors into an estimated radiation-field error relative to the source's own field (0.2 maps to $q = 1.5$). The first error is a jump in the stored acceleration, seen as the largest second difference $\Delta^2\mathbf{a}$ over the four interpolation knots; the cubic smooths such a jump across its light sphere. The second is stored acceleration inconsistent with the change of stored velocity across the segment ($d\boldsymbol{\beta}/dR = -\mathbf{a}$). The indicator reports $q = -1$ when the retarded point lies beyond the outermost shell or the segment reaches past the ring-buffer depth, and it raises q inside the first shell and on shells damped by the optional Nyquist taper. The app hatches pixels with $q > 1.5$ or $q < 0$. The thresholds are heuristic (Section 4.3); the indicator never changes field values.

3 Verification protocol

Table 2 comes from `tests/gpu/table2_v6_2.mjs` (raw data in `results/table2_v6_2.tsv` and `.json`). Tables 3–4, the straight-line and targeted limiter tests and the v6.0.0 cost figures come from `tests/gpu/tables.mjs` (raw data in `results/tables.json`, version-6 kernel). The app's own suite for all four kernels (v6f, v6f precise, version 6, v5) is in `results/suite_swiftshader_v6_2.tsv`. The v6f attribution, bracket-check and field-floor studies are in `validation/f32newton/`; the version-5 error attribution, the limiter statistics and the far-field estimates are in `validation/analysis/`; the PyCharge comparison is in `validation/pycharge/`.

1. **Reference.** The exact field is computed in float64 directly on the analytic trajectory (32-step bracketed bisection on t_{ret}, then up to 12 safeguarded Newton steps) with $\epsilon = 0$. With softening off, a Python port of this reference agrees with PyCharge 2.0.1 [7], which obtains velocity and acceleration by automatic differentiation of the position. The largest relative difference is 1.3×10^{-11} (circular orbits with $\beta \le 0.99$, harmonic oscillation, straight-line motion). The reference is not externally checked above $\gamma = 7.1$.

2. **Benchmark.** A charge on a circular orbit of radius 0.85 ($c = 2.35$). A ring buffer ($\Delta t = 0.00675$) is seeded and advanced 16 steps, and the shells are populated from it. Observers: 256 points at fixed distance from the orbit centre crossing the pulse emitted tangentially at retarded radius 1.15 (half-width $\max(10^{-6}, 4\gamma^{-3})$ rad), and a 64-point ring at radius 1.15.

3. **Readback.** Fields are read back from an `RGBA32F` framebuffer (`gl.readPixels`). The tables use WebGL2 with the SwiftShader software renderer, which implements IEEE float32. With the v6.0.0 kernel, the in-app suite on an Apple M1 Max (Metal) reproduced the software-rendered pulse-sweep column to within run-to-run scatter (e.g. 0.0089% and 0.059% at $\gamma = 22.4$ and 70.7, $M_T = 512$; `results/suite_m1max.tsv`). Version 6 at commit `06edf87` gave 0.01% and 0.03% (two decimals; `results/suite_m1max_web_kform.txt`). For the v6.2.0 kernels, the in-app suite on the same M1 Max in Safari (development builds with the kernel code of v6.2.0), run with v6f and with v6f precise, printed identical values for both: pulse-sweep errors of 0.01, 0.02, 0.04, 0.13 and 0.40% at $M_T = 128$ ($\beta = 0.8$ to 0.9999), at most 0.02% at $M_T = 512$ and 0.00% at $M_T = 2048$, and at $\gamma = 70.7$ 0.02%, 0.01% and 0.00% at $M_T = 512$, 1024 and 2048, equal to the float64 spline (0.0236%, 0.00602%, 0.00166%) to the printed digits

(`results/suite_m1max_v6_2.txt`). Its ring-buffer histories do not exercise the field-floor fix (Section 2.6). Other GPUs should be checked with the suite.

4. **Regression.** The single-file solver deposited with version 5 is kept in the repository. The version-5 column in Table 2 is that deposited shader run on the same history, against the unsoftened reference. In v6.2.0 the v5 and v6 programs give bit-identical readback and rendered frames to their reference copies (v6: commit `904bcce`), and the v6 columns of Table 2 reproduce `results/tables.json` exactly.

4 Results

4.1 Accuracy across Lorentz factors

			sweep						ring			
				df64		**df64**	df64	v5				obs.
β	γ	M_T	v6	v6	**v6f**	**v6f**	v6fp	alg.	v6	v6f	v6fp	rounding
0.8	1.67	128	0.0067%	0.0067%	0.0067%	0.0067%	0.0067%	0.0067%	0.0029%	0.0029%	0.0029%	2.2×10^{-5}%
0.8	1.67	512	4.4×10^{-4}%	4.4×10^{-4}%	4.4×10^{-4}%	4.4×10^{-4}%	4.4×10^{-4}%	0.0010%	1.5×10^{-4}%	1.5×10^{-4}%	1.5×10^{-4}%	
0.8	1.67	2048	3.5×10^{-5}%	3.6×10^{-5}%	3.7×10^{-5}%	3.7×10^{-5}%	3.7×10^{-5}%	0.0034%	2.2×10^{-5}%	2.1×10^{-5}%	2.1×10^{-5}%	
0.95	3.20	128	0.017%	0.017%	0.017%	0.017%	0.017%	0.017%	0.12%	0.12%	0.12%	3.1×10^{-4}%
0.95	3.20	512	0.0010%	0.0010%	0.0010%	0.0010%	0.0010%	0.0019%	0.0068%	0.0069%	0.0069%	
0.95	3.20	2048	6.7×10^{-5}%	6.8×10^{-5}%	6.6×10^{-5}%	6.9×10^{-5}%	6.9×10^{-5}%	0.0075%	5.2×10^{-4}%	5.4×10^{-4}%	5.4×10^{-4}%	
0.99	7.09	128	0.038%	0.038%	0.038%	0.038%	0.038%	0.039%	0.012%	0.012%	0.012%	0.0034%
0.99	7.09	512	0.0022%	0.0022%	0.0022%	0.0022%	0.0022%	0.0097%	8.4×10^{-4}%	8.6×10^{-4}%	8.2×10^{-4}%	
0.99	7.09	2048	1.8×10^{-4}%	1.6×10^{-4}%	1.6×10^{-4}%	1.6×10^{-4}%	1.5×10^{-4}%	0.015%	1.0×10^{-4}%	1.1×10^{-4}%	5.7×10^{-5}%	
0.999	22.4	128	0.13%	0.13%	0.13%	0.13%	0.13%	0.26%	0.013%	0.013%	0.013%	0.11%
0.999	22.4	512	0.0077%	0.0079%	**0.0076%**	0.0076%	0.0076%	0.22%	0.0016%	0.0017%	0.0017%	
0.999	22.4	2048	5.4×10^{-4}%	5.8×10^{-4}%	5.5×10^{-4}%	5.4×10^{-4}%	4.8×10^{-4}%	0.29%	2.5×10^{-4}%	1.7×10^{-4}%	2.0×10^{-4}%	
0.9999	70.7	128	0.37%	0.43%	0.39%	0.40%	0.40%	48%	0.045%	0.045%	0.045%	3.2%
0.9999	70.7	512	0.036%	0.049%	**0.023%**	0.024%	0.024%	7.6%	0.0028%	0.0026%	0.0028%	
0.9999	70.7	2048	0.0070%	0.0081%	0.0018%	0.0016%	0.0016%	7.8%	7.0×10^{-5}%	1.3×10^{-4}%	1.4×10^{-4}%	
0.99999	224	128	—	6.8%	—	0.58%	0.58%	—	0.041%	0.041%	0.041%	> 100%
0.99999	224	512	—	1.2%	—	**0.066%**	0.067%	—	0.0011%	0.0010%	0.0011%	
0.99999	224	2048	—	0.22%	—	**0.0060%**	0.0050%	—	6.0×10^{-5}%	1.1×10^{-4}%	3.6×10^{-5}%	

Table 2: Relative L_2 error of GPU readback against the exact LW field (no softening), ring-buffer history, library readback in software rendering. Sweep: 256 observers crossing the synchrotron pulse; ring: 64 observers at radius 1.15. Columns without "df64" round observers to float32 and compare with the exact field at the rounded point; "df64" places them with double-float coordinates. At $\gamma = 224$ float32 observer coordinates cannot resolve the pulse (last column: the error from rounding the observers alone), so only the df64 columns are meaningful there. "v6" is the version-6 kernel of v6.2.0, with Eq. (4) and the Newton fix; it differs from the v6.0.0 kernel reported in version 6.1. v6fp: v6f precise, i.e. v6f with the field-floor fix; its float32-observer sweep is in the TSV. v5 alg.: the version-5 algorithm (float32 root function, Theorem 1 limiter, bisection on the spline) was not designed for $\gamma > 100$.

At $\gamma \leq 7$ the three kernels agree to float32 noise, and the remaining error is interpolation of the history, which converges with M_T. At $\gamma = 70.7$, v6f lowers the sweep error 1.6× at $M_T = 512$ and 4.0× at $M_T = 2048$. At $\gamma = 224$ the gain is 18× at $M_T = 512$ and 37× at $M_T = 2048$ (v6f precise: 17× and 45×). On these ring-buffer histories the field-floor fix changes little. These gains are the chord-deficit floor of Section 2.4. With it removed, the ring-buffer sweep at $\gamma = 70.7$ equals the float64 evaluation of the same spline, and at $\gamma = 224$ it too is limited by interpolation of the history. Relative to the version-5 solver, v6f's error at $M_T \geq 512$ is 29–520× lower at $\gamma = 22.4$ and 330–4400× lower at $\gamma = 70.7$; at $M_T = 128$, $\gamma = 70.7$, where the version-5 limiter was active across the pulse, it falls from 48% to 0.39%. There are small regressions. The first is in the $\gamma = 70.7$, $M_T = 128$ sweep with float32 observers: 0.37% to 0.39%. That row is limited by interpolation, and with double-float observers it improves (0.43% to 0.40%). The others are in the 64-point ring at $M_T = 2048$, at the float32 level of that test: at $\gamma = 70.7$, 7.0×10^{-5}% to 1.3×10^{-4}% (v6f precise 1.4×10^{-4}%), and at $\gamma = 224$, 6.0×10^{-5}% to 1.1×10^{-4}% (v6f precise 3.6×10^{-5}%).

4.2 Radial grid at equal memory

128 shells, $R \in [0, 220]$	sweep	ring	spacing at $R = 1.15$	first spacing
Stereographic $L \tan(\theta/2)$, $L = 1.35$	**0.038%**	0.012%	0.029	0.017
Logarithmic $r_0(e^{\alpha u} - 1)$	0.33%	0.080%	0.081	0.0035
Quadratic $R_{\max} u^2$	1.6%	0.69%	0.27	0.027
Uniform	$> 100\%$	$> 100\%$	1.72	1.72

Table 3: Same memory and reach, $\beta = 0.99$, prefilter off for every grid (version 5 applied it only to the stereographic grid, and its quadratic-grid entry of 0.21% does not reproduce). The advantage follows from the near-uniform spacing of Eq. (5) for $R \lesssim L$; other parameters for the competing grids may narrow the gap. Version-6 kernel; the app's suite gives the same sweep values with v6f (0.0384%, 0.330%, 1.60%).

4.3 Far field and the validity indicator

M_T (outermost shell)	$R = 1.15$	$R = 3$	$R = 10$	$R = 30$	$R = 100$	$R = 200$
128 (220)	0.0063% (0)	0.61% (1)	2.9% (64)	$> 100\%$ (64)	$> 100\%$ (64)	$> 100\%$ (64)
512 (880)	0.0015% (0)	0.011% (0)	0.20% (29)	31% (64)	$> 100\%$ (64)	$> 100\%$ (64)
2048 (3520)	6.4×10^{-5}% (0)	0.0019% (0)	0.010% (0)	0.18% (64)	$> 100\%$ (64)	$> 100\%$ (64)

Table 4: Circular orbit, $\beta = 0.9$, **version-6 kernel**: relative L_2 error on rings of radius R around the orbit centre, and in parentheses the number of the 64 observers marked by the validity indicator of Section 2.8. The first column gives the outermost shell; the ring buffer holds 260 length units of history. Across all far-field and straight-line cases run ($\beta = 0.5$ and 0.9; $M_T = 128$, 512, 2048; taper on and off), the L_2 error over unmarked observers on a ring is at most 0.37%, and the worst single unmarked observer is 0.68%. v6f (`results/tables_far_v6f.json`; v6f precise gives identical values, since $\gamma \le 2.3$ is below the fix's gate) gives the same flag counts and agrees to 1% in every cell but one, $R = 1.15$, $M_T = 2048$, where it is worse (1.4×10^{-4}%). The table is set by interpolation of the history.

Accuracy falls with distance once the shell spacing, which grows as R^2, no longer resolves the orbital motion; the stored history reaches far beyond the radius at which it stays accurate. The indicator is conservative. For unaccelerated motion only its ring-buffer test applies: with $\beta = 0.5$ straight-line motion and observers 50–800 units away, every observer with a retarded point beyond the buffer is marked (errors 2–69%), and every unmarked one is accurate to 3×10^{-5}%. Shells modified by the optional Nyquist taper are marked.

Dragged histories. Before the acceleration terms were added, the indicator marked 0–35% of the observers with error above 1% on dragged histories (a pointer path with jumps, holds and reversals; exact field from the recorded history). It now marks 99%, 96% and 99% of them at $M_T = 128$, 512 and 2048, and the 99th-percentile error of unmarked observers is below 1% at every M_T. On smooth presets the new terms add at most 0.5% marked observers.

4.4 When the causality limiter matters

On smooth histories resampled onto the shells, the unlimited cubic rarely exceeds c. For hyperbolic motion ($\gamma \approx 11$ at the observation time) it reaches $\|\mathbf{z}'\| = 1.057$, but only on the outermost shell ($R \in [73, 220]$, $M_T = 128$). Observers whose retarded points fall in that segment see 73% and 22% error (two runs) with the squared-speed certificate, 96% and 49% with Theorem 1, and 93%

and 68% with no limiter; the indicator marks all of them. On under-resolved segments a limiter can also raise the error. The certificate's role is a guarantee of one root per observer, at no cost on resolved histories; it does not make an under-resolved history accurate.

4.5 Cost

On an Apple M1 Max GPU, with the v6.0.0 kernel, a 256×256 grid takes 5–7 ms for one source and 20 ms for 16 sources at $M_T = 2048$, including upload and readback (`validation/benchmarks/results_webgl_m1max.json`). For the v6.2 kernels on the same GPU see Table 5. Cost is proportional to the number of observers times the number of sources, as for any direct sum of LW fields. There is no hierarchical (multipole-type) acceleration, so the method suits few sources and many observation points.

kernel	compiled with	fps	vs. v6
v5 (original)	—	120	(likely display-capped)
v6	—	43	1
v6f (default)	`F32N`, `F32N_NOFIX`	42	0.98
v6f precise	`F32N`	33	0.77

Table 5: Frame rate on an Apple M1 Max (Safari, WebGL2 on Metal), 2D app, 32 sources, default scene, known path (`results/suite_m1max_v6_2.txt`). In SwiftShader v6f takes about half the frame time of v6; on the M1 it does not.

4.6 Comparison with PyCharge

To separate the algorithm from the language, we ported the method to JAX (float64) and ran it against PyCharge 2.0.1 [7] on the same CPU, the same problems (charges on circular orbits of radius 0.85) and the same 256×256 grid, against the exact field (`validation/benchmarks/`). PyCharge was run both with its default solver and with the cheapest fixed-point/Newton budgets that converge on the full grid.

case (65,536 points)	PyCharge, default	PyCharge, cheapest converged	this method (JAX), $M_T = 2048$	its error
$\beta = 0.9$, 1 source	0.27 s	0.040 s	0.038 s	1.1×10^{-6}
$\beta = 0.99$, 1 source	0.28 s	0.066 s	0.061 s	6.2×10^{-6}
$\beta = 0.999$, 1 source	0.53 s	0.18 s	0.034 s	2.4×10^{-5}
$\beta = 0.9999$, 1 source	0.33 s	0.34 s	0.039 s	2.9×10^{-5}
$\beta = 0.9$, 16 sources	5.1 s	0.61 s	0.66 s	1.0×10^{-6}

Table 6: Float64 on a 2-core CPU, medians of three runs after compilation. PyCharge is exact when it converges (10^{-11}–10^{-14}); the method here interpolates a sampled history. Errors from the v6.0.0 JAX port; with the Newton fix most fall slightly and the $\beta = 0.999$ entry rises 1.4–1.55$\times$ (`validation/benchmarks/README.md`).

The two are tied for $\gamma \lesssim 7$ and for 16 sources; at $\gamma = 22$ and 71 the method is 5$\times$ and 9$\times$ faster, because PyCharge's fixed-point iteration needs more steps as $\beta \to 1$. The trade is accuracy: a few times 10^{-5} instead of machine precision. PyCharge also requires float64, which consumer GPUs run slowly and Apple GPUs do not support. Forced to float32, PyCharge's error on the pulse sweep is 0.027%, 0.86% and 27% at $\gamma = 7.1$, 22.4 and 70.7. For the float32 WebGL kernel v6f ($M_T = 512$) it is 0.0022%, 0.0076% and 0.023%. PyCharge also handles 3D and self-consistent dynamics, which this method does not. This method works from any sampled history rather than a differentiable analytic trajectory.

5 Limitations and scope

1. **Two dimensions and prescribed motion.** Charges move in a plane and do not respond to each other's fields; there is no radiation reaction. The retarded-time solve, the certificate (dot products only) and the LW expression do not depend on the dimension, but motion in 3D is not implemented or tested here.

2. **History, not far-field storage.** The shells are a resampled view of an append-only ring buffer. Coarsening old history removes the radiation pulse. A hand-off to a stored far field would only be valid beyond $R \gtrsim 1.75\,\gamma\rho/\varepsilon$ for relative error ε (ρ the radius of curvature), after removing the Fresnel arrival delay; without that correction the 1% radius is $\approx 210\,\rho\gamma^3$, and storage would cost more than the worldline. (Version 5's far-field hand-off and Hopf-bundle polarization proposals are withdrawn; a transverse field stored as a Cartesian vector orthogonal to $\hat{\mathbf{n}}$ needs no bundle, as in Cauchy-characteristic extraction [22].)

3. **Very high γ.** The method is tested to $\gamma = 224$. Above $\gamma \approx 100$ observers must be placed with double-float coordinates, which is available through the readback API but not in the rendered view. On exactly sampled paths v6f is limited by float32 rounding of directions (about 2×10^{-3}% at $\gamma = 224$), and v6f precise by float32 storage of **u** and **a**; on ring-buffer histories the error is set by interpolation of the history.

4. **Hardware.** The accuracy tables use an IEEE float32 software renderer. One GPU was checked: an Apple M1 Max, with version 6 as v6.0.0 and at commit `06edf87`, and with both v6.2.0 kernels (development builds with the same kernel code). The incremental root carries no compiler barrier. On the M1 (Safari, Metal) the in-app suite run with v6f and with v6f precise matched the float64 spline at $\gamma = 70.7$ to the printed digits (Section 3), so this float32 code survived Metal's reassociation there. The suite's ring-buffer histories do not exercise the field-floor fix, whose exact products are barrier-guarded. Other compilers may transform floating-point code in ways the barrier does not anticipate; the probe page and the in-app suite check this on the user's GPU.

5. **Sources faster than the wave speed.** In media or acoustics, sources can outrun the wave and observers see several retarded roots. The certificate does not apply there; counting all roots per segment is a natural extension.

6 Changes from version 6.1

The error attribution of Section 2.4 is corrected, and the new default kernel v6f (Section 2.6) removes the floor it identifies; the optional v6f precise adds the field-floor fix; v6 and v5 remain selectable and unchanged. Software changes since v6.0.0 are described in Sections 2.5–2.8: the radiation term of Eq. (4), the Newton safeguard, the display path, physical dragging, known-path mode and the indicator's acceleration terms. Table 2 is remeasured, Tables 1 and 5 are new, and Table 4 reflects the new indicator. Citations are added to Ryne *et al.*, Zhao *et al.*, Nakayama *et al.*, Shintake's real-time animation, RaDiO, SRW and Boyd.

7 Changes from version 5

- The "exact" reference in version 5 applied the Plummer softening ($\epsilon = 0.06$); all errors here are against the unsoftened field.

- Version 5 attributed the single-precision floor to κ^{-3} evaluation and Newton conditioning; for that algorithm the measured cause is the root-function subtraction and absolute knot storage (Section 2.4). Its 6–9% GPU-versus-CPU discrepancy is of the size of legal float32 reorderings (5.7–7.3%).

- The Theorem 1 limiter, which caused most of the $\gamma = 70.7$, $M_T = 128$ error, is replaced by Proposition 1. Version 5's Float64 convergence column ran without a limiter.
- Version 5's far-field result used a preset at $0.44c$ with no acceleration that jumped back every 4.4 time units; it is replaced by Table 4.
- Some version-5 entries do not reproduce with its own code (quadratic grid 4.4% vs 0.21%; $\beta = 0.999$, $M_T = 128$: 0.25% vs 2.47%; the max $|\Delta \mathbf{z}|$ column; two far-field cells were not computed).
- The ensemble preset double-counted the frame rotation in its acceleration and rescaled velocity without position; both are fixed. The far-field hand-off and the Hopf-bundle proposal are withdrawn (Section 5).

8 Code and data availability

Every number in this paper not marked as from v6.0.0 was produced with the kernel code of **hopf-hough v6.2.0** (git tag `v6.2.0`, commit **[TODO: commit hash]**), archived at **[TODO: Zenodo software DOI for v6.2.0]**: by its tests and scripts, or, for the M1 Max figures, by development builds with identical kernel code (provenance per figure in `results/suite_m1max_v6_2.txt`). That release (MIT License) contains the tool, library, tests, raw results and the scripts that produce every number here; the app is a single HTML file. Numbers marked v6.0.0 come from **hopf-hough v6.0.0** (tag `v6.0.0`, commit `d87f823`), archived at doi:10.5281/zenodo.23601496, which version 6.1 of this paper cites. Development continues at `https://github.com/timsdsdio/hopf-hough`; results should be reproduced from the tagged release.

Acknowledgments and AI assistance

The core idea (applying Hough-transform-style voting over the Hopf/stereographic structure to retarded-time solving) and the research direction are the author's; the author designed and steered the multi-agent workflow, and AI agents did the implementation and contributed specific technical ideas, credited below. AI systems contributed substantially, and all results were produced by code in the repository and checked by the author:

- **Gemini Argon (Google)**: the original WebGL2 shader, the Theorem 1 construction, the cancellation-free kernel, the version-5 verification suite and drafts of earlier versions.
- **Gemini (Google), as an outside reviewer**: the incremental form of the in-segment root function (Section 2.6) began from its suggestion; the working form, with the chord deficit from the CPU and the denominator corrected for the shader's time direction, was found and corrected in testing.
- **Astra (OpenAI)**: adversarial review of earlier drafts and code (six rounds), including the pulse-crossing sweep and the matched-reach grid benchmark.
- **Claude (Anthropic), Claude Fable 5.1 and Claude Opus 5.5**: review of version 5; the multi-agent analysis behind the squared-speed certificate, the version-5 floor and its double-float fix, and the far-field, prior-art and preset corrections; the TypeScript port, production shader, tests, PyCharge cross-check, JAX benchmark and documentation; the validity-indicator audit; and the diagnosis of the Metal compiler undoing the double-float arithmetic, with the barrier that fixes it. For version 6.2: the radiation-term and Newton fixes, the indicator's acceleration terms, physical dragging, known-path mode and the cheaper display path; the attribution of the version-6 floor to the chord deficit, the v6f kernel and the field-floor fix, and their port to the app and library; and drafting and editing of this version.

References

[1] J. D. Jackson, *Classical Electrodynamics*, 3rd ed. (Wiley, 1999), Ch. 14.

[2] J.-L. Vay *et al.*, "Warp-X: A new exascale computing platform for beam–plasma simulations," *Nucl. Instrum. Methods Phys. Res. A* **909**, 476–479 (2018).

[3] M. Pardal, A. Sainte-Marie, A. Reboul-Salze, R. A. Fonseca and J. Vieira, "RaDiO: an efficient spatiotemporal radiation diagnostic for particle-in-cell codes," *Comput. Phys. Commun.* **285**, 108634 (2023), doi:10.1016/j.cpc.2022.108634; arXiv:2302.14812.

[4] T. Shintake, "New real-time simulation technique for synchrotron and undulator radiations," in *Proc. LINAC 2002*, Gyeongju, Korea, pp. 661–663 (2002).

[5] T. Shintake, "New mathematical method for radiation field of moving charge," in *Proc. EPAC 2002*, Paris, paper WEPRI038, pp. 1667–1669 (2002).

[6] C.-K. Huang, F. Li, H. N. Rakotoarivelo, B. Shen, J. Domine, B. E. Carlsten, G. A. Dilts, R. V. Garimella, T. J. T. Kwan and R. W. Robey, "CoSyR: A novel beam dynamics code for the modeling of synchrotron radiation effects," *Nucl. Instrum. Methods Phys. Res. A* **1034**, 166808 (2022), doi:10.1016/j.nima.2022.166808; arXiv:2109.14901; code at https://github.com/lanl/cosyr.

[7] M. Filipovich and S. Hughes, "PyCharge: An open-source Python package for self-consistent electrodynamics simulations of Lorentz oscillators and moving point charges," *Comput. Phys. Commun.* **274**, 108291 (2022).

[8] M. Filipovich and S. Hughes, "Space-time computation and visualization of the electromagnetic fields and potentials generated by moving point charges," *Am. J. Phys.* **89**, 482–489 (2021).

[9] R. D. Ryne, C. E. Mitchell, J. Qiang, B. Carlsten and N. Yampolsky, "Using a Liénard–Wiechert solver to study coherent synchrotron radiation effects," in *Proc. FEL2013*, New York, NY, USA, paper MOOCNO04, pp. 17–23 (2013).

[10] Y. Zhao, Y. Liang, Y. Zhang, X. Ma and H. Liu, "An electromagnetic particle-particle method for relativistic electron bunch dynamics from early expansion to long-range transport," arXiv:2605.05543 (2026).

[11] D. Nakayama, K. Oda and K. Yasuda, "Seeing through the light cone: Visualizing electromagnetic fields in special relativity," arXiv:2505.20596 (2025).

[12] O. Chubar and P. Elleaume, "Accurate and efficient computation of synchrotron radiation in the near field region," in *Proc. EPAC'98*, Stockholm, paper THP01G, pp. 1177–1179 (1998).

[13] A. Zenginoğlu, "Hyperboloidal foliations and scri-fixing," *Class. Quantum Grav.* **25**, 145002 (2008).

[14] J. P. Boyd, "Orthogonal rational functions on a semi-infinite interval," *J. Comput. Phys.* **70**, 63–88 (1987), doi:10.1016/0021-9991(87)90002-7.

[15] K. S. Brentner, "Numerical algorithms for acoustic integrals — the devil is in the details," AIAA Paper 96-1706 (1996).

[16] K. S. Brentner, "An efficient and robust method for predicting helicopter high-speed impulsive noise," *J. Sound Vib.* **203**, 87–100 (1997).

[17] R. T. Farouki, *Pythagorean-Hodograph Curves: Algebra and Geometry Inseparable* (Springer, 2008).

[18] J. Tordesillas and J. P. How, "MINVO basis: Finding simplexes with minimum volume enclosing polynomial curves," arXiv:2010.10726 (2020).

[19] F. N. Fritsch and R. E. Carlson, "Monotone piecewise cubic interpolation," *SIAM J. Numer. Anal.* **17**, 238–246 (1980).

[20] A. Thall, "Extended-precision floating-point numbers for GPU computation," ACM SIGGRAPH Research Posters (2006).

[21] G. Da Graça and D. Defour, "Implementation of float-float operators on graphics hardware," arXiv:cs/0603115 (2006).

[22] N. T. Bishop, R. Gómez, L. Lehner, M. Maharaj and J. Winicour, "High-powered gravitational news," *Phys. Rev. D* **56**, 6298 (1997).

[23] Analysis scripts, saved outputs and reports in the software repository: `validation/analysis/` (version-5 error attribution, limiter study, far-field estimates) and `validation/f32newton/` (v6f chord-deficit attribution, float32 bracket check, field-floor attribution and fix).